

Contents

1	Mathematical Foundations of GAMLSS	1
1.1	1. Distribution Derivatives	1
1.2	2. Special Functions	10
1.3	3. Numerical Stability and Implementation	10
1.4	4. GAMLSS Iteration (P-IRLS)	11
1.5	5. Smoothing and Penalty Matrices	14
1.6	6. Effective Degrees of Freedom	17
1.7	7. Link Functions	17
1.8	8. GCV Gradient for Smoothing Parameter Optimization	18
1.9	9. Mean-Variance Relationships	23
1.10	10. Convergence and Initialization	24
1.11	11. Model Selection and Diagnostics	25
1.12	12. Code Map	27
1.13	13. Notation Glossary	28
1.14	References	29

1 Mathematical Foundations of GAMLSS

This document provides detailed mathematical derivations for the GAMLSS implementation in **glissando**. It covers distribution-specific derivatives, special functions, numerical algorithms, and convergence theory.

Generated PDF: run `bash docs/build-math-pdf.sh` to produce `docs/mathematics.pdf` (a numbered, hyperlinked, table-of-contents'd PDF). The PDF build uses `pandoc + xelatex`; see the script header for details.

1.1 1. Distribution Derivatives

This section provides detailed derivations of score functions (first derivatives of log-likelihood) and Fisher information (expected second derivatives) for all implemented distributions. These quantities drive the penalized iteratively reweighted least squares (P-IRLS) algorithm.

Key quantities: - **Score** $u = \frac{\partial \ell}{\partial \eta}$: Direction of steepest ascent for the log-likelihood - **Fisher information** $w = I_{\eta}$: Expected curvature (used as weight in IRLS) - **Working response** $z = \eta + u/w$: Adjusted target for the weighted least squares problem

For each distribution, we derive these quantities accounting for the link function via the chain rule.

1.1.1 1.1 Gaussian Distribution

Parameterization: $Y \sim N(\mu, \sigma^2)$

Parameters: - μ (mean): identity link - σ (standard deviation): log link

Variance: $\text{Var}(Y) = \sigma^2$

1.1.1.1 Log-Likelihood

$$\ell(\mu, \sigma | y) = -\frac{1}{2} \log(2\pi) - \log(\sigma) - \frac{(y - \mu)^2}{2\sigma^2}$$

1.1.1.2 Derivatives for μ (identity link) Score:

$$\frac{\partial \ell}{\partial \mu} = \frac{y - \mu}{\sigma^2}$$

Fisher information:

$$I_\mu = \mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \mu^2} \right] = \frac{1}{\sigma^2}$$

Since we use identity link ($\eta = \mu$), no chain rule needed:

$$u_\mu = \frac{y - \mu}{\sigma^2}, \quad w_\mu = \frac{1}{\sigma^2}$$

1.1.1.3 Derivatives for σ (log link) Let $\eta_\sigma = \log(\sigma)$, so $\sigma = e^{\eta_\sigma}$.

Score with respect to σ :

$$\frac{\partial \ell}{\partial \sigma} = -\frac{1}{\sigma} + \frac{(y - \mu)^2}{\sigma^3}$$

Chain rule for log link:

$$\frac{\partial \ell}{\partial \eta_\sigma} = \sigma \cdot \frac{\partial \ell}{\partial \sigma} = -1 + \frac{(y - \mu)^2}{\sigma^2} = \frac{(y - \mu)^2 - \sigma^2}{\sigma^2}$$

Fisher information on η_σ scale:

$$I_{\eta_\sigma} = \mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \eta_\sigma^2} \right] = 2$$

This comes from $\mathbb{E}[(Y - \mu)^2/\sigma^2] = 1$ and $\text{Var}[(Y - \mu)^2/\sigma^2] = 2$ for Gaussian.

Implementation:

$$u_\sigma = \frac{(y - \mu)^2 - \sigma^2}{\sigma^2}, \quad w_\sigma = 2$$

1.1.2 1.2 Student-t Distribution

1.1.3 Log-Likelihood

For observation y with parameters μ (location), σ (scale), and ν (degrees of freedom):

$$\ell(\mu, \sigma, \nu | y) = \log \Gamma \left(\frac{\nu + 1}{2} \right) - \log \Gamma \left(\frac{\nu}{2} \right) - \frac{1}{2} \log(\nu \pi \sigma^2) - \frac{\nu + 1}{2} \log \left(1 + \frac{z^2}{\nu} \right)$$

where $z = \frac{y - \mu}{\sigma}$.

1.1.4 First Derivatives (Score Functions)

Define the robustified weight:

$$w = \frac{\nu + 1}{\nu + z^2}$$

1.1.4.1 Location parameter μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{w \cdot z}{\sigma}$$

1.1.4.2 Scale parameter σ (log link):

$$\frac{\partial \ell}{\partial \log(\sigma)} = w \cdot z^2 - 1$$

1.1.4.3 Degrees of freedom ν (log link):

$$\frac{\partial \ell}{\partial \log(\nu)} = \nu \left[\frac{1}{2} \left(\psi \left(\frac{\nu+1}{2} \right) - \psi \left(\frac{\nu}{2} \right) - \log \left(1 + \frac{z^2}{\nu} \right) + \frac{w \cdot z^2 - 1}{\nu} \right) \right]$$

where $\psi(x) = \frac{d}{dx} \log \Gamma(x)$ is the digamma function.

1.1.5 Expected Fisher Information

1.1.5.1 For μ :

$$I_\mu = \mathbb{E} \left[\frac{w}{\sigma^2} \right] = \frac{\nu+1}{\sigma^2(\nu+3)}$$

In practice, we use the observed information (plug in w directly).

1.1.5.2 For σ (log link):

$$I_{\log(\sigma)} = \frac{2\nu}{\nu+3}$$

1.1.5.3 For ν (log link):

$$I_{\log(\nu)} = \frac{\nu^2}{4} \left[\psi' \left(\frac{\nu}{2} \right) - \psi' \left(\frac{\nu+1}{2} \right) + \frac{2(\nu+3)}{\nu(\nu+1)} \right]$$

where $\psi'(x)$ is the trigamma function.

1.1.6 Numerical Considerations

1. **Minimum ν :** $\nu > 2$ is enforced to ensure finite variance, via a *floored log link* on ν ($\eta = \log \nu$, $\nu = \max(e^\eta, 2)$). The optimizer can explore the heavy-tail region without the variance $\sigma^2 \nu / (\nu - 2)$ becoming undefined; the floor never binds when the true ν is well above 2. See `FlooredLogLink` in `src/distributions/links.rs`.
2. **Minimum σ :** Enforce $\sigma \geq 10^{-6}$ to prevent division by zero
3. **Weight clamping:** The denominator $\nu + z^2$ should be $\geq 10^{-10}$
4. **Information positivity:** Ensure $I_{\log(\nu)} \geq 10^{-6}$

1.1.7 1.3 Poisson Distribution

Parameterization: $Y \sim \text{Poisson}(\mu)$

Parameters: - μ (rate/mean): log link

Variance: $\text{Var}(Y) = \mu$ (equidispersion)

1.1.7.1 Log-Likelihood

$$\ell(\mu|y) = y \log(\mu) - \mu - \log(y!)$$

1.1.7.2 Derivatives for μ (log link)

Let $\eta = \log(\mu)$, so $\mu = e^\eta$.

Score with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{y}{\mu} - 1 = \frac{y - \mu}{\mu}$$

Chain rule for log link:

$$\frac{\partial \ell}{\partial \eta} = \mu \cdot \frac{\partial \ell}{\partial \mu} = y - \mu$$

Fisher information on η scale:

$$I_\eta = \mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \eta^2} \right] = \mu$$

Implementation:

$$u_\mu = y - \mu, \quad w_\mu = \mu$$

1.1.8 1.4 Gamma Distribution

Parameterization: $Y \sim \text{Gamma}(\alpha, \theta)$ where $\alpha = 1/\sigma^2$ (shape), $\theta = \mu\sigma^2$ (scale)

Parameters: - μ (mean): log link - σ (coefficient of variation): log link

Variance: $\text{Var}(Y) = \mu^2 \sigma^2$

Mean-CV relationship: $\text{CV}(Y) = \sigma$

1.1.8.1 Log-Likelihood

$$\ell(\mu, \sigma|y) = -\alpha \log(\theta) - \log \Gamma(\alpha) + (\alpha - 1) \log(y) - \frac{y}{\theta}$$

where $\alpha = 1/\sigma^2$ and $\theta = \mu\sigma^2$.

Simplifying:

$$\ell = -\frac{1}{\sigma^2} \log(\mu\sigma^2) - \log \Gamma\left(\frac{1}{\sigma^2}\right) + \left(\frac{1}{\sigma^2} - 1\right) \log(y) - \frac{y}{\mu\sigma^2}$$

1.1.8.2 Derivatives for μ (log link) Let $\eta_\mu = \log(\mu)$.

Score with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = -\frac{\alpha}{\mu} + \frac{y}{\mu^2 \theta} = -\frac{1}{\mu\sigma^2} + \frac{y}{\mu^2 \sigma^2} = \frac{y - \mu}{\mu^2 \sigma^2}$$

Chain rule for log link:

$$\frac{\partial \ell}{\partial \eta_\mu} = \mu \cdot \frac{\partial \ell}{\partial \mu} = \frac{y - \mu}{\mu \sigma^2}$$

Fisher information on η_μ scale:

$$I_{\eta_\mu} = \frac{1}{\sigma^2}$$

Implementation:

$$u_\mu = \frac{y - \mu}{\mu \sigma^2}, \quad w_\mu = \frac{1}{\sigma^2}$$

1.1.8.3 Derivatives for σ (log link) Let $\eta_\sigma = \log(\sigma)$.

Score with respect to σ :

$$\frac{\partial \ell}{\partial \sigma} = -\frac{2\alpha}{\sigma} + \frac{2\alpha}{\sigma} \psi(\alpha) + \frac{2\alpha}{\sigma} \log(y) - \frac{2\alpha}{\sigma} \log(\theta) + \frac{2y}{\mu \sigma^3}$$

where $\psi(x) = \frac{d}{dx} \log \Gamma(x)$ is the digamma function.

Chain rule for log link ($\frac{d\sigma}{d\eta_\sigma} = \sigma$):

$$\frac{\partial \ell}{\partial \eta_\sigma} = \frac{2}{\sigma^2} \left[\psi\left(\frac{1}{\sigma^2}\right) + 2\log(\sigma) - \log\left(\frac{y}{\mu}\right) + \frac{y}{\mu} - 1 \right]$$

Fisher information involves trigamma $\psi'(x) = \frac{d^2}{dx^2} \log \Gamma(x)$:

$$I_{\eta_\sigma} = \frac{4}{\sigma^4} \psi'\left(\frac{1}{\sigma^2}\right) - \frac{2}{\sigma^2}$$

Implementation:

$$u_\sigma = \frac{2}{\sigma^2} [\psi(\alpha) + 2\log(\sigma) - \log(y/\mu) + y/\mu - 1], \quad w_\sigma = \frac{4}{\sigma^4} \psi'(\alpha) - \frac{2}{\sigma^2}$$

1.1.9 1.5 Negative Binomial Distribution

Parameterization: $Y \sim \text{NB}(r, p)$ with mean μ and dispersion σ (NB2 parameterization)

Parameters: - μ (mean): log link - σ (overdispersion): log link

Variance: $\text{Var}(Y) = \mu + \sigma\mu^2$ (overdispersion relative to Poisson)

Relationship to standard NB: $r = 1/\sigma$, $p = 1/(1 + \sigma\mu)$

1.1.9.1 Log-Likelihood

$$\ell(\mu, \sigma|y) = \log \Gamma(y + r) - \log \Gamma(r) - \log(y!) + r \log(p) + y \log(1 - p)$$

where $r = 1/\sigma$ and $p = 1/(1 + \sigma\mu)$.

Simplifying in terms of (μ, σ) :

$$\ell = \log \Gamma(y + 1/\sigma) - \log \Gamma(1/\sigma) - \log(y!) + \frac{1}{\sigma} \log\left(\frac{1}{1 + \sigma\mu}\right) + y \log\left(\frac{\sigma\mu}{1 + \sigma\mu}\right)$$

1.1.9.2 Derivatives for μ (log link) Let $\eta_\mu = \log(\mu)$.

Score with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{y - \mu}{1 + \sigma\mu}$$

Chain rule for log link ($\frac{d\mu}{d\eta_\mu} = \mu$):

$$\frac{\partial \ell}{\partial \eta_\mu} = \mu \cdot \frac{\partial \ell}{\partial \mu} = \frac{\mu(y - \mu)}{1 + \sigma\mu}$$

However, in IRLS we use the observed Fisher information $w_\mu = \frac{\mu}{1 + \sigma\mu}$, giving working response:

$$u_\mu = \frac{y - \mu}{1 + \sigma\mu}, \quad w_\mu = \frac{\mu}{1 + \sigma\mu}$$

This follows from the score being proportional to observed weight, as noted in the full derivation in Rigby & Stasinopoulos (2005).

Why observed information here: for μ in NB the expected information $I_{\eta_\mu} = \mathbb{E}[\mu(Y - \mu)/(1 + \sigma\mu)^2]$ collapses to $\mu/(1 + \sigma\mu)$ after expectation, which coincides numerically with the observed-information form

because of the variance identity $\text{Var}(Y) = \mu(1 + \sigma\mu)$. Rigby & Stasinopoulos (2005, eq. (15)) use this simplification; `src/distributions/negative_binomial.rs` follows it. As $\sigma \rightarrow 0$ the weight reduces to μ — exactly the Poisson Fisher weight — confirming the limiting Poisson behavior.

Implementation (for IRLS):

$$u_\mu = \frac{y - \mu}{1 + \sigma\mu}, \quad w_\mu = \frac{\mu}{1 + \sigma\mu}$$

1.1.9.3 Derivatives for σ (log link) Let $\eta_\sigma = \log(\sigma)$ and $r = 1/\sigma$.

Score with respect to σ :

$$\frac{\partial \ell}{\partial \sigma} = -\frac{1}{\sigma^2} \left[\psi(y + r) - \psi(r) - \log(1 + \sigma\mu) + \frac{y - \mu}{1 + \sigma\mu} \right]$$

Chain rule for log link:

$$\frac{\partial \ell}{\partial \eta_\sigma} = \sigma \cdot \frac{\partial \ell}{\partial \sigma} = -\frac{1}{\sigma} \left[\psi(y + r) - \psi(r) - \log(1 + \sigma\mu) + \frac{y - \mu}{1 + \sigma\mu} \right]$$

Fisher information (using approximation):

$$I_{\eta_\sigma} = \frac{1}{\sigma^2} \psi'(r)$$

Implementation:

$$u_\sigma = -\frac{1}{\sigma} \left[\psi(y + 1/\sigma) - \psi(1/\sigma) - \log(1 + \sigma\mu) + \frac{y - \mu}{1 + \sigma\mu} \right], \quad w_\sigma = \frac{\psi'(1/\sigma)}{\sigma^2}$$

1.1.10 1.6 Beta Distribution

Parameterization: $Y \sim \text{Beta}(\alpha, \beta)$ where $\alpha = \mu\phi$ and $\beta = (1 - \mu)\phi$

Parameters: - μ (mean): logit link - ϕ (precision): log link

Variance: $\text{Var}(Y) = \frac{\mu(1-\mu)}{1+\phi}$

1.1.10.1 Log-Likelihood

$$\ell(\mu, \phi | y) = \log \Gamma(\phi) - \log \Gamma(\alpha) - \log \Gamma(\beta) + (\alpha - 1) \log(y) + (\beta - 1) \log(1 - y)$$

where $\alpha = \mu\phi$ and $\beta = (1 - \mu)\phi$.

1.1.10.2 Derivatives for μ (logit link) Let $\eta_\mu = \text{logit}(\mu) = \log(\mu/(1 - \mu))$.

Score with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \phi [\log(y) - \log(1 - y) - \psi(\alpha) + \psi(\beta)]$$

Chain rule for logit link ($\frac{d\mu}{d\eta_\mu} = \mu(1 - \mu)$):

$$\frac{\partial \ell}{\partial \eta_\mu} = \mu(1 - \mu) \cdot \frac{\partial \ell}{\partial \mu} = \mu(1 - \mu)\phi [\log(y) - \log(1 - y) - \psi(\alpha) + \psi(\beta)]$$

Fisher information:

$$I_{\eta_\mu} = [\mu(1-\mu)]^2 \phi^2 [\psi'(\alpha) + \psi'(\beta)]$$

Implementation:

$$\begin{aligned} u_\mu &= \mu(1-\mu)\phi [\log(y) - \log(1-y) - \psi(\mu\phi) + \psi((1-\mu)\phi)] \\ w_\mu &= [\mu(1-\mu)]^2 \phi^2 [\psi'(\mu\phi) + \psi'((1-\mu)\phi)] \end{aligned}$$

1.1.10.3 Derivatives for ϕ (log link) Let $\eta_\phi = \log(\phi)$.

Score with respect to ϕ :

$$\frac{\partial \ell}{\partial \phi} = \psi(\phi) - \mu\psi(\alpha) - (1-\mu)\psi(\beta) + \mu \log(y) + (1-\mu) \log(1-y)$$

Chain rule for log link:

$$\frac{\partial \ell}{\partial \eta_\phi} = \phi \cdot \frac{\partial \ell}{\partial \phi}$$

Fisher information:

$$I_{\eta_\phi} = \phi^2 [\psi'(\phi) - \mu^2 \psi'(\alpha) - (1-\mu)^2 \psi'(\beta)]$$

Implementation:

$$\begin{aligned} u_\phi &= \phi [\psi(\phi) - \mu\psi(\mu\phi) - (1-\mu)\psi((1-\mu)\phi) + \mu \log(y) + (1-\mu) \log(1-y)] \\ w_\phi &= \phi^2 [\psi'(\phi) - \mu^2 \psi'(\mu\phi) - (1-\mu)^2 \psi'((1-\mu)\phi)] \end{aligned}$$

1.1.11 1.7 Binomial Distribution

Parameterization: $Y \sim \text{Binomial}(n, \mu)$ where Y is the number of successes in n trials

Parameters: - μ (probability): logit link

Variance: $\text{Var}(Y) = n\mu(1-\mu)$

1.1.11.1 Log-Likelihood

$$\ell(\mu|y) = y \log(\mu) + (n-y) \log(1-\mu) + \log \binom{n}{y}$$

1.1.11.2 Derivatives for μ (logit link) Let $\eta = \text{logit}(\mu)$.

Score with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{y}{\mu} - \frac{n-y}{1-\mu} = \frac{y-n\mu}{\mu(1-\mu)}$$

Chain rule for logit link ($\frac{d\mu}{d\eta} = \mu(1-\mu)$):

$$\frac{\partial \ell}{\partial \eta} = \mu(1-\mu) \cdot \frac{\partial \ell}{\partial \mu} = y - n\mu$$

Fisher information on η scale:

$$I_\eta = n\mu(1-\mu)$$

Implementation:

$$u_\mu = y - n\mu, \quad w_\mu = n\mu(1-\mu)$$

1.1.12 1.8 Ordered Categorical Distribution (0cat)

Parameterization: Proportional-odds / cumulative-logit for ordinal response $y \in \{1, \dots, R\}$, $R \in \{2, 3, 4, 5\}$.

Parameters: - μ (latent linear predictor): identity link - δ_1 (first threshold θ_1): identity link - δ_k (positive increment $\theta_k - \theta_{k-1}$), $k = 2, \dots, R-1$: log link

Threshold reconstruction: $\theta_1 = \delta_1$ (unconstrained); $\theta_k = \theta_{k-1} + \exp(\delta_k)$ for $k \geq 2$, ensuring strict monotonicity $\theta_1 < \dots < \theta_{R-1}$.

1.1.12.1 Cumulative-Logit Probabilities Define the logistic CDF (argument clamped to $[-30, 30]$) and density:

$$F_k = \text{logistic}(\theta_k - \mu) = \frac{1}{1 + e^{-(\theta_k - \mu)}}, \quad f_k = F_k(1 - F_k)$$

with boundary conventions $F_0 = 0$, $F_R = 1$. Category probabilities:

$$\pi_r = F_r - F_{r-1}, \quad r = 1, \dots, R.$$

Each π_r is floored at $\text{MIN_PROB} = 10^{-10}$ and the vector is renormalized to sum to 1.

1.1.12.2 Log-Likelihood

$$\ell_i = \log \pi_{y_i} = \log(F_{y_i} - F_{y_i-1})$$

1.1.12.3 Derivatives for μ (identity link) Using boundary conventions $f_0 = f_R = 0$, for observation $y_i = r$:

Score:

$$u_\mu = \frac{f_{r-1} - f_r}{\pi_r}$$

Expected Fisher information (sum over all categories):

$$w_\mu = \sum_{r=1}^R \frac{(f_{r-1} - f_r)^2}{\pi_r}$$

Implementation:

$$u_\mu = \frac{f_{r-1} - f_r}{\pi_r}, \quad w_\mu = \sum_{r=1}^R \frac{(f_{r-1} - f_r)^2}{\pi_r}$$

Both floored at MIN_WEIGHT where needed.

1.1.12.4 Derivatives for δ_k (identity link for $k = 1$, log link for $k \geq 2$) The Jacobian of the $\eta_k \rightarrow \theta_k$ map is:

$$\text{jac}_k = \begin{cases} 1 & k = 1 \text{ (identity link)} \\ \exp(\eta_k) = \delta_k & k \geq 2 \text{ (log link; stored as response-scale value)} \end{cases}$$

For observation $y_i = r$, threshold δ_k only affects π_r when $r \geq k$:

Score:

$$u_k = \begin{cases} 0 & r < k \\ \frac{\text{jac}_k (f_r \mathbf{1}\{r \leq R-1\} - f_{r-1} \mathbf{1}\{r > k\})}{\pi_r} & r \geq k \end{cases}$$

Expected Fisher information (sum over categories $r = k, \dots, R$):

$$w_k = \sum_{r=k}^R \frac{\left(\text{jac}_k(f_r \mathbf{1}\{r \leq R-1\} - f_{r-1} \mathbf{1}\{r > k\}) \right)^2}{\pi_r}$$

floored at MIN_WEIGHT.

1.1.12.5 Moments

$$\mathbb{E}[Y] = \sum_{r=1}^R r \pi_r, \quad \text{Var}(Y) = \sum_{r=1}^R r^2 \pi_r - (\mathbb{E}[Y])^2$$

(variance clamped to ≥ 0).

Implementation note: `Ocat` carries `n_categories` state and cannot be recovered from a name string alone. It is excluded from `from_name` and from the WASM / JSON string-dispatch routes. Construct it explicitly in Rust or via the Python `Ocat(n_categories=R)` class. The fitting loop treats each threshold δ_k as an intercept-only formula — no new solver machinery is required beyond what drives any other scalar distribution parameter.

Source: `src/distributions/ocat.rs`.

1.1.13 1.9 Example: Computing Derivatives for Gaussian Data

Suppose we have data $y = [2.1, 1.9, 3.2]$ and current parameter estimates $\mu = [2.0, 2.0, 3.0]$, $\sigma = [0.5, 0.5, 0.5]$.

For μ (identity link):

Residuals: $r = y - \mu = [0.1, -0.1, 0.2]$

Weights: $w_\mu = 1/\sigma^2 = [4.0, 4.0, 4.0]$

Score: $u_\mu = r \cdot w_\mu = [0.4, -0.4, 0.8]$

Working response: $z_\mu = \mu + u_\mu/w_\mu = \mu + r = y$

(For identity link with constant weights, the working response is just y)

For σ (log link):

Squared residuals: $r^2 = [0.01, 0.01, 0.04]$

Score on $\eta_\sigma = \log(\sigma)$ scale:

$$u_\sigma = \frac{r^2 - \sigma^2}{\sigma^2} = \frac{[0.01, 0.01, 0.04] - 0.25}{0.25} = [-0.96, -0.96, -0.84]$$

Weights: $w_\sigma = [2.0, 2.0, 2.0]$

Working response on log scale:

$$z_\sigma = \log(\sigma) + u_\sigma/w_\sigma = [-0.693, -0.693, -0.693] + [-0.48, -0.48, -0.42] = [-1.17, -1.17, -1.11]$$

The PWLS solver then regresses z_σ on the design matrix for σ with weights w_σ to update σ .

1.2 2. Special Functions

1.2.1 2.1 Digamma Function

The digamma function $\psi(x) = \frac{d}{dx} \log \Gamma(x) = \frac{\Gamma'(x)}{\Gamma(x)}$ appears in derivatives for Gamma, Negative Binomial, and Beta distributions.

1.2.1.1 Recurrence Relation For $x < 5$, use:

$$\psi(x) = \psi(x+1) - \frac{1}{x}$$

Repeatedly apply until $x \geq 5$.

1.2.1.2 Asymptotic Expansion For $x \geq 5$, use the series (Abramowitz & Stegun 6.3.18):

$$\psi(x) \sim \log(x) - \frac{1}{2x} - \frac{1}{12x^2} + \frac{1}{120x^4} - \frac{1}{252x^6} + O(x^{-8})$$

1.2.1.3 Known Values (for testing)

- $\psi(1) = -\gamma \approx -0.5772156649$ (Euler-Mascheroni constant)
- $\psi(2) = 1 - \gamma \approx 0.4227843351$
- $\psi(1/2) = -\gamma - 2\log(2) \approx -1.9635100260$

1.2.2 2.2 Trigamma Function

The trigamma function $\psi'(x) = \frac{d^2}{dx^2} \log \Gamma(x)$ is needed for Student-t Fisher information.

1.2.3 Recurrence Relation

For $x < 5$, use:

$$\psi'(x) = \psi'(x+1) + \frac{1}{x^2}$$

1.2.4 Asymptotic Expansion

For $x \geq 5$, use the series (Abramowitz & Stegun 6.4.11):

$$\psi'(x) \sim \frac{1}{x} + \frac{1}{2x^2} + \frac{1}{6x^3} - \frac{1}{30x^5} + \frac{1}{42x^7} - \frac{1}{30x^9} + O(x^{-11})$$

1.2.5 Known Values (for testing)

- $\psi'(1) = \frac{\pi^2}{6} \approx 1.644934067$
 - $\psi'(2) = \frac{\pi^2}{6} - 1 \approx 0.644934067$
 - $\psi'(1/2) = \frac{\pi^2}{2} \approx 4.934802201$
-

1.3 3. Numerical Stability and Implementation

1.3.1 3.1 Parameter Bounds

To prevent numerical issues, parameters are clamped to safe ranges:

Minimum positive value: 10^{-10} - Used for: μ (when must be positive), σ , ϕ , probabilities

Minimum weight: 10^{-6} - Fisher information weights are floored at this value to ensure positive definiteness of the weight matrix

Link function bounds: - Log/logit links: $\eta \in [-30, 30]$ - Prevents overflow in $\exp(\eta)$ (since $e^{30} \approx 10^{13}$ and $e^{-30} \approx 10^{-14}$)

1.3.2 3.2 Distribution-Specific Safeguards

Student-t: - Enforce $\nu > 2$ (finite variance) via the floored log link on ν (§7) - Ensure $\nu + z^2 \geq 10^{-10}$ to prevent division issues in robustifying weight

Gamma: - Clamp σ to reasonable range to prevent extreme shape parameters

Beta: - Clamp y to $(10^{-10}, 1 - 10^{-10})$ before computing $\log(y)$ and $\log(1 - y)$ - Clamp μ similarly before computing logit

Negative Binomial: - Ensure $1 + \sigma\mu$ stays positive and bounded away from zero

Ordered Categorical: - Floor each category probability π_r at $\text{MIN_PROB} = 10^{-10}$ before computing scores and Fisher info - Renormalize: divide all π_r by their sum after flooring, so that $\sum_r \pi_r = 1$ is maintained

1.3.3 3.3 Batched Special Function Computation

Digamma and trigamma functions are computed in batched vectorized form for performance:

- For $n < 10,000$ observations: sequential computation
- For $n \geq 10,000$: parallel computation via Rayon (when `parallel` feature enabled)

The switchpoint is empirically tuned to balance overhead vs. speedup.

1.4 4. GAMLSS Iteration (P-IRLS)

1.4.1 4.1 Backfitting Outer Loop

GAMLSS fits the joint model by cycling through distribution parameters one at a time, holding the others fixed (Rigby & Stasinopoulos 2005, §3). For a P -parameter family (e.g. Gaussian: $P = 2$; Student-t: $P = 3$):

```
for cycle in 0..max_iterations:
  all_converged = true
  for each parameter theta_k in family.parameters():
    beta_k_old = beta_k
    beta_k      = p_irls_step(theta_k | other params held fixed)
    delta       = max_abs(beta_k - beta_k_old)
    scale       = max(max_abs(beta_k), 1.0) # floor at 1 for 0(1) coefficients
    if delta / scale >= tolerance:
      all_converged = false
  if all_converged:
    break
```

Source: `src/fitting/mod.rs`. The convergence criterion is **per-parameter relative**: each distribution parameter θ_k must independently satisfy

$$\frac{\|\beta_k^{(t+1)} - \beta_k^{(t)}\|_\infty}{\max(\|\beta_k^{(t+1)}\|_\infty, 1)} < \epsilon,$$

and all parameters must pass before the outer loop terminates. Using each parameter's own scale prevents a large-coefficient parameter (e.g. μ when data are un-normalized) from dominating the denominator and

masking drift in a small-coefficient parameter (e.g. a log-scale σ whose intercept is near 0). The floor of 1 in the denominator keeps the test equivalent to an absolute threshold when all coefficients are $O(1)$.

The active default is $\epsilon = 10^{-3}$, max iterations = 200 (DEFAULT_TOLERANCE, DEFAULT_MAX_ITER at `src/fitting/mod.rs:31-32`).

1.4.2 4.2 Inner P-IRLS Step (Fisher Scoring)

For each parameter θ_k (e.g., μ , σ , ν):

1. **Working response:**

$$z_k = \eta_k + \frac{u_k}{w_k}$$

where $u_k = \frac{\partial \ell}{\partial \eta_k}$ and w_k is the (expected, or in some cases observed) Fisher information.

2. **Penalized weighted least squares** (see §8.1 for the Cholesky solve):

$$\hat{\beta}_k = \arg \min_{\beta} \|W_k^{1/2}(z_k - X_k \beta)\|^2 + \sum_j \lambda_j \beta^T S_j \beta$$

3. **Update linear predictor:**

$$\eta_k^{(\text{new})} = X_k \hat{\beta}_k$$

4. **Smoothing parameter selection** — GCV, REML, or Fellner-Schall (§§ 8, 8.2).

Source: `src/fitting/scoring.rs:40-180`.

1.4.3 4.3 Working-Response Derivation

Why is $z = \eta + u/w$ the right pseudo-response? Expand the log-likelihood for one observation in η around the current iterate η_0 :

$$\ell(\eta) \approx \ell(\eta_0) + u(\eta_0) (\eta - \eta_0) - \frac{1}{2} w(\eta_0) (\eta - \eta_0)^2$$

where $u = \partial \ell / \partial \eta$ and $w = -\mathbb{E}[\partial^2 \ell / \partial \eta^2]$ (Fisher information, positive). Maximizing the quadratic Taylor surrogate over η gives the one-Newton-step update $\eta - \eta_0 = u/w$. Setting $\eta = X\beta$ and stacking observations yields the Gauss-Newton normal equations:

$$X^T W X \Delta \beta = X^T W (u/w)$$

with $\Delta \beta = \beta - \beta_0$ and the diagonal weight $W = \text{diag}(w_i)$. Equivalently, regress the **working response** $z = \eta_0 + u/w$ on X with weights W :

$$X^T W X \beta = X^T W z.$$

Adding the penalty $\sum_j \lambda_j \beta^T S_j \beta$ to the surrogate produces the PWLS system of §8.1. This is exactly the IRLS construction of McCullagh & Nelder (1989) generalized to one distributional parameter at a time.

1.4.4 4.4 Numerical Safeguards in the Inner Loop

Cataloged here so the magic constants are auditable. All apply per observation, per inner iteration. Source: `src/fitting/scoring.rs`, `src/distributions/links.rs`.

Symbol	Value	Where	Purpose
MIN_WEIGHT	10^{-6}	scoring.rs:28	Floor on w_i so the weight matrix stays positive definite; near-zero Fisher info would blow up u/w . Hits counted in weight_floor_hits .
MAX_STEP	20.0	scoring.rs:32	Clamps u_i/w_i to ± 20 in η -units. Prevents wild Newton steps from outliers. Hits counted in step_cap_hits .
MAX_ETA, MIN_ETA	± 30	links.rs:10-12	Clamps η before applying the inverse log/logit link so $\exp(\eta)$ stays finite ($e^{30} \approx 10^{13}$).
MIN_POSITIVE	10^{-10}	links.rs:8, diagnostics.rs:15	Floor on parameters that must be strictly positive (σ , ϕ , probabilities), and on variances before the square-root in residual computation.

Persistent non-zero **weight_floor_hits** or **step_cap_hits** at convergence signals model misspecification or ill-conditioned data; transient hits in early iterations are normal.

1.4.5 4.5 Prior / Observation Weights

Each call to the P-IRLS **step** function accepts an optional **prior_weights**: `Option<Array1<f64>>` — a per-observation scale applied to each observation’s likelihood contribution. When absent, a ones vector is substituted (uniform weights).

Let $\text{safe_w}_i = \max(w_i, \text{MIN_WEIGHT})$ where w_i is the distribution’s Fisher information for observation i .

Working response — unchanged by prior weights:

$$z_i = \eta_i + \text{clip}\left(\frac{u_i}{\text{safe_w}_i}, \pm \text{MAX_STEP}\right)$$

The step size $u_i/\text{safe_w}_i$ is a property of log-likelihood curvature and must not be scaled by the observation weight.

Solve weight — carries the prior factor:

$$W_{ii} = \text{prior}_i \cdot \text{safe_w}_i$$

Effect on the penalized normal equations: because z divides by safe_w while W multiplies by $\text{prior} \cdot \text{safe_w}$, the prior weight scales each observation’s contribution to both $X^T W X$ and $X^T W z$ exactly once — standard frequency / precision-weight semantics without distorting the IRLS step length:

$$(X^T W X + S_\lambda) \hat{\beta} = X^T W z, \quad W = \text{diag}(\text{prior}_i \cdot \text{safe_w}_i).$$

When **prior_weights** = `None`, the formula reduces to $W = \text{diag}(\text{safe_w}_i)$, identical to the unweighted case.

Source: `src/fitting/scoring.rs:61-141 (step)`.

1.5 5. Smoothing and Penalty Matrices

1.5.1 5.0 B-Spline Basis (Cox–de Boor)

A degree- p B-spline basis with k functions is defined on a knot vector

$$\tau_0 < \tau_1 < \dots < \tau_{k+p}.$$

Knot placement (P-spline, bs="ps"). The implementation uses **equally-spaced** knots — the requirement of the Eilers–Marx P-spline, because the difference penalty $D^T D$ only approximates a roughness integral when the basis knots are uniform:

$$\tau_j = \min(x) + (j - p) \Delta, \quad \Delta = \frac{\max(x) - \min(x)}{k - p}, \quad j = 0, \dots, k + p.$$

This places $\tau_p = \min(x)$ and $\tau_k = \max(x)$, with p extra knots extending beyond each end of the data range. Source: `src/splines.rs (select_knots)`.

The **Cox–de Boor recursion** evaluates the basis stably (Piegl & Tiller 1997, Algorithm A2.2). Let i be the **knot span** — the index satisfying $\tau_i \leq x < \tau_{i+1}$, clamped to $[p, k - 1]$. Initialize $N_0 = 1$. Then for $j = 1, \dots, p$:

$$\text{left}[j] = x - \tau_{i+1-j}, \quad \text{right}[j] = \tau_{i+j} - x,$$

and for $r = 0, \dots, j - 1$:

$$\begin{aligned} \text{denom} &= \text{right}[r + 1] + \text{left}[j - r], & \text{term} &= N_r / \text{denom}, \\ N_r &\leftarrow N_r^{\text{saved}} + \text{right}[r + 1] \cdot \text{term}, & N_r^{\text{saved}} &\leftarrow \text{left}[j - r] \cdot \text{term}. \end{aligned}$$

At the end of iteration j , $N_0, \dots, N_j$ hold the $j + 1$ non-zero degree- j basis values at x . **Critical implementation note:** `left[j]` and `right[j]` must be computed *once, before the inner r -loop*, so that the full arrays `left[1..j]` and `right[1..j]` remain valid when the inner loop reads `left[j - r]` and `right[r + 1]`. Computing them inside the r -loop overwrites earlier slots and produces wrong interior basis values for degree ≥ 3 (the endpoint functions are unaffected but the middle ones are wrong; both the correct and the broken form still satisfy partition-of-unity, so property-only tests do not catch the error).

After the full degree- p pass, $N_0, \dots, N_p$ are the $p + 1$ non-zero values at x ; they map to columns $[i - p, i]$ of the basis matrix. Source: `src/splines.rs (create_basis_matrix, evaluate_basis_functions_into, find_knot_span)`.

Properties:

- **Partition of unity:** $\sum_{j=1}^k B_j(x_i) = 1$ for every x_i in the interior of the knot range.
- **Non-negativity:** $B_j(x) \geq 0$.
- **Local support:** $B_{i,p}$ is non-zero only on $[\tau_i, \tau_{i+p+1}]$, so each row of the design matrix has at most $p + 1$ non-zero entries.

1.5.2 5.1 Sum-to-Zero Reparameterization (Identifiability)

Partition-of-unity creates a rank deficiency when the model has both an intercept and a smooth term: the column vector $\mathbf{1}_n$ lies in the column space of B , so the design matrix $[\mathbf{1}_n \mid B]$ is column-rank-deficient. To restore identifiability, the smooth is reparameterized through an orthonormal basis of the subspace orthogonal to $\mathbf{1}_k$.

The implementation uses a **Householder reflector** (`src/splines.rs:1-39`). Choose

$$v = \mathbf{1}_k + \sqrt{k} e_1$$

which reflects $\mathbf{1}_k$ onto $-\sqrt{k} e_1$. Form

$$H = I_k - \frac{2}{\|v\|^2} vv^T, \quad Z = H_{:,2:k}$$

i.e. drop the first column of H . Then $Z \in \mathbb{R}^{k \times (k-1)}$ satisfies

$$Z^T Z = I_{k-1}, \quad Z^T \mathbf{1}_k = 0.$$

Reparameterize $\beta = Z\gamma$ to get the constrained basis and penalty:

$$B_{\text{new}} = BZ, \quad S_{\text{new}} = Z^T S Z.$$

The new design matrix $[\mathbf{1}_n \mid BZ]$ has full column rank, and the constraint $\mathbf{1}_k^T \beta = 0$ — i.e. the smooth has mean zero across knots — is enforced automatically.

1.5.3 P-Spline Penalty (Eilers & Marx 1996)

For 2nd-order differences:

$$S = D^T D$$

where D is the $(m-2) \times m$ difference matrix:

$$D = \begin{bmatrix} 1 & -2 & 1 & 0 & \cdots & 0 \\ 0 & 1 & -2 & 1 & \cdots & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & \cdots & 0 & 1 & -2 & 1 \end{bmatrix}$$

This penalizes curvature: $\lambda \beta^T S \beta \approx \lambda \int (\beta''(x))^2 dx$

1.5.4 Tensor Product Smooth and Anisotropic Penalty

For a bivariate smooth $f(x_1, x_2)$ built from marginal bases $B_1 \in \mathbb{R}^{n \times k_1}$ and $B_2 \in \mathbb{R}^{n \times k_2}$, the joint basis is the **row-wise Kronecker product** of the marginals:

$$B[i, :] = B_1[i, :] \otimes B_2[i, :] \in \mathbb{R}^{k_1 k_2}, \quad B \in \mathbb{R}^{n \times (k_1 k_2)}.$$

The coefficient vector $\beta \in \mathbb{R}^{k_1 k_2}$ is indexed lexicographically so $\beta_{(i-1)k_2+j}$ multiplies $B_1[\cdot, i] \cdot B_2[\cdot, j]$.

Anisotropic penalties are obtained by Kronecker'ing each marginal difference penalty with an identity matrix:

$$S^{(1)} = S_1 \otimes I_{k_2}, \quad S^{(2)} = I_{k_1} \otimes S_2$$

with two independent smoothing parameters:

$$\lambda_1 \beta^T S^{(1)} \beta + \lambda_2 \beta^T S^{(2)} \beta.$$

$S^{(1)}$ penalizes roughness in the x_1 direction at every x_2 slice; $S^{(2)}$ vice versa. Choosing two λ 's lets the GCV/REML routine pick different smoothness for each margin (Wood 2017, §5.6).

The sum-to-zero constraint of §5.1 is applied **per margin** before the Kronecker product when an intercept is present:

$$B_{1,\text{new}} = B_1 Z_1, \quad B_{2,\text{new}} = B_2 Z_2, \quad S^{(1)} = (Z_1^T S_1 Z_1) \otimes I_{k_2-1}, \quad S^{(2)} = I_{k_1-1} \otimes (Z_2^T S_2 Z_2).$$

Source: `src/fitting/assembler.rs:44-102`, `src/splines.rs:8-41`.

1.5.5 Random Effect Penalty

For an observation-to-group map $g : \{1, \dots, n\} \rightarrow \{1, \dots, G\}$, the design matrix is the binary indicator

$$Z \in \{0, 1\}^{n \times G}, \quad Z[i, g] = \mathbf{1}\{g(i) = g\}.$$

Coupled with the **identity penalty** $S = I_G$, the model

$$\eta = \mathbf{1}\beta_0 + Z\alpha, \quad \lambda\alpha^T\alpha = \lambda \sum_{g=1}^G \alpha_g^2$$

is equivalent to the Bayesian random-intercept prior $\alpha_g \sim N(0, 1/\lambda)$ — the ridge-penalized empirical Bayes interpretation. Since each row of Z sums to 1, the sum-to-zero reparameterization of §5.1 is again applied when an intercept is present:

$$Z_{\text{new}} = Z Z_c \in \mathbb{R}^{n \times (G-1)}, \quad Z_c^T I_G Z_c = I_{G-1}.$$

Source: `src/fitting/assembler.rs:103-126`.

1.5.6 5.1 Block-Sparse Penalty Matrices

In practice, penalty matrices S_j are often **block-sparse**: they only penalize a subset of the coefficient vector. For example, if the model has an intercept, linear terms, and a smooth term:

$$\beta = [\beta_{\text{intercept}}, \beta_{\text{linear}}, \beta_{\text{smooth}}]^T$$

The penalty for the smooth term is:

$$S_{\text{smooth}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & S_{\text{block}} \end{bmatrix}$$

where S_{block} is the $(n_{\text{splines}} \times n_{\text{splines}})$ penalty for the spline coefficients.

Efficient Storage: Instead of storing the full $p \times p$ matrix (mostly zeros), we store: - **block**: The non-zero sub-matrix S_{block} - **offset**: Starting index in the full coefficient vector - **full_dim**: Total dimension p

Operations:

Scaled addition: $A \leftarrow A + \lambda S$

$$A[i : i + k, j : j + k] \leftarrow A[i : i + k, j : j + k] + \lambda \cdot S_{\text{block}}$$

where $i = \text{offset}$ and $k = \text{dim}(S_{\text{block}})$.

Matrix-vector product: $v = S\beta$

$$v[i : i + k] = S_{\text{block}} \cdot \beta[i : i + k], \quad v[\text{elsewhere}] = 0$$

This exploits sparsity for computational efficiency, especially important when many terms have independent penalties.

1.6 6. Effective Degrees of Freedom

The “wiggleness” of the fit is measured by:

$$\text{EDF} = \text{tr}(H)$$

where $H = X(X^T W X + \lambda S)^{-1} X^T W X$ is the hat matrix.

Interpretation: - EDF ≈ 2 for a linear fit (intercept + slope) - EDF $\approx$ number of basis functions for an unpenalized spline - EDF between these extremes reflects smoothing

1.7 7. Link Functions

1.7.1 Identity Link

- **Function:** $g(\mu) = \mu$
- **Inverse:** $g^{-1}(\eta) = \eta$
- **Domain:** $\mu \in \mathbb{R}$
- **Use:** Gaussian mean, Student-t location
- **Derivative:** $\frac{d\mu}{d\eta} = 1$

1.7.2 Log Link

- **Function:** $g(\mu) = \log(\mu)$
- **Inverse:** $g^{-1}(\eta) = \exp(\eta)$
- **Domain:** $\mu > 0$
- **Use:** Poisson rate, Gamma mean, scale parameters (σ)
- **Derivative:** $\frac{d\mu}{d\eta} = \mu$
- **Numerical safeguard:** Clamp $\eta \in [-30, 30]$ to prevent overflow

1.7.3 Floored Log Link

- **Function:** $g(\mu) = \log(\max(\mu, c))$
- **Inverse:** $g^{-1}(\eta) = \max(\exp(\eta), c)$
- **Domain:** $\mu \geq c$
- **Use:** Student-t degrees of freedom ν with floor $c = 2$, keeping the variance $\sigma^2 \nu / (\nu - 2)$ finite as the optimizer explores the heavy-tail region
- **Derivative:** $\frac{d\mu}{d\eta} = \mu$ above the floor (the floor binds only transiently and never when the true ν is well above 2)
- **Numerical safeguard:** same $\eta \in [-30, 30]$ clamp as the log link, plus the lower floor c . See `FlooredLogLink` in `src/distributions/links.rs`.

1.7.4 Logit Link

- **Function:** $g(\mu) = \log\left(\frac{\mu}{1-\mu}\right)$
- **Inverse:** $g^{-1}(\eta) = \frac{1}{1+e^{-\eta}} = \frac{e^{\eta}}{1+e^{\eta}}$
- **Domain:** $\mu \in (0, 1)$
- **Use:** Binomial probability, Beta mean
- **Derivative:** $\frac{d\mu}{d\eta} = \mu(1-\mu)$
- **Numerical safeguards:**
 - Clamp $\mu \in [10^{-10}, 1 - 10^{-10}]$ before applying link
 - Clamp $\eta \in [-30, 30]$ to prevent overflow in inverse

1.7.5 Chain Rule for Link Functions

When modeling parameter θ with link function g , we have $\eta = g(\theta)$. The score and Fisher information transform as:

$$\frac{\partial \ell}{\partial \eta} = \frac{d\theta}{d\eta} \cdot \frac{\partial \ell}{\partial \theta}$$

$$I_\eta = \left(\frac{d\theta}{d\eta} \right)^2 I_\theta$$

For log link ($\eta = \log(\theta)$):

$$\frac{d\theta}{d\eta} = \theta, \quad \frac{\partial \ell}{\partial \eta} = \theta \cdot \frac{\partial \ell}{\partial \theta}, \quad I_\eta = \theta^2 I_\theta$$

For logit link ($\eta = \text{logit}(\theta)$):

$$\frac{d\theta}{d\eta} = \theta(1-\theta), \quad \frac{\partial \ell}{\partial \eta} = \theta(1-\theta) \cdot \frac{\partial \ell}{\partial \theta}, \quad I_\eta = [\theta(1-\theta)]^2 I_\theta$$

For identity link ($\eta = \theta$):

$$\frac{d\theta}{d\eta} = 1, \quad \frac{\partial \ell}{\partial \eta} = \frac{\partial \ell}{\partial \theta}, \quad I_\eta = I_\theta$$

1.7.6 Working Response and IRLS

The working response in the IRLS algorithm is:

$$z = \eta + \frac{u}{w}$$

where $u = \frac{\partial \ell}{\partial \eta}$ and $w = I_\eta$.

For a Poisson model with log link: - Current: $\eta = \log(\hat{\mu})$ - Score: $u = y - \hat{\mu}$ - Weight: $w = \hat{\mu}$ - Working response: $z = \log(\hat{\mu}) + \frac{y - \hat{\mu}}{\hat{\mu}}$

For a Binomial model with logit link: - Current: $\eta = \text{logit}(\hat{\mu})$ - Score: $u = y - n\hat{\mu}$ - Weight: $w = n\hat{\mu}(1 - \hat{\mu})$ - Working response: $z = \text{logit}(\hat{\mu}) + \frac{y - n\hat{\mu}}{n\hat{\mu}(1 - \hat{\mu})}$

The PWLS problem then becomes:

$$\hat{\beta}^{(t+1)} = \arg \min_{\beta} \sum_i w_i (z_i - x_i^T \beta)^2 + \sum_j \lambda_j \beta^T S_j \beta$$

1.8 8. GCV Gradient for Smoothing Parameter Optimization

The GCV score is minimized using L-BFGS optimization. The gradient with respect to $\log(\lambda_j)$ requires careful derivation.

1.8.1 Setup

Let $V = (X^T W X + \sum_j \lambda_j S_j)^{-1}$ and $\hat{\beta}(\lambda) = V X^T W z$.

1.8.2 Key Derivatives

Derivative of β with respect to λ_j :

$$\frac{\partial \hat{\beta}}{\partial \lambda_j} = -VS_j\hat{\beta}$$

Derivative of RSS:

$$\frac{\partial \text{RSS}}{\partial \lambda_j} = 2(X^T W r)^T VS_j\hat{\beta}$$

where $r = z - X\hat{\beta}$ are the residuals.

Derivative of EDF:

$$\frac{\partial \text{EDF}}{\partial \lambda_j} = -\text{tr}(VS_jVX^T W X)$$

1.8.3 Full GCV Gradient

Using the quotient rule on $\text{GCV} = \frac{n \cdot \text{RSS}}{(n - \text{EDF})^2}$:

$$\frac{\partial \text{GCV}}{\partial \lambda_j} = \frac{n}{(n - \text{EDF})^3} \left[\frac{\partial \text{RSS}}{\partial \lambda_j} (n - \text{EDF}) + 2 \cdot \text{RSS} \cdot \frac{\partial \text{EDF}}{\partial \lambda_j} \right]$$

1.8.4 Chain Rule for Log-Space

Since we optimize in log-space:

$$\frac{\partial \text{GCV}}{\partial \log(\lambda_j)} = \lambda_j \cdot \frac{\partial \text{GCV}}{\partial \lambda_j}$$

This ensures $\lambda_j > 0$ without explicit constraints.

1.8.5 8.1 Efficient PWLS Solution via Cholesky Decomposition

The penalized weighted least squares problem:

$$\hat{\beta} = \arg \min_{\beta} \|W^{1/2}(z - X\beta)\|^2 + \sum_j \lambda_j \beta^T S_j \beta$$

has the normal equations:

$$(X^T W X + S_{\lambda})\hat{\beta} = X^T W z$$

where $S_{\lambda} = \sum_j \lambda_j S_j$.

Key observation: $X^T W X + S_{\lambda}$ is symmetric positive definite (SPD) when S_j are positive semi-definite and W has positive diagonal entries.

1.8.5.1 Cholesky Approach

1. Form the weighted design matrix: $X_w = W^{1/2} X$
2. Form the coefficient matrix: $A = X_w^T X_w + S_{\lambda}$
3. Compute Cholesky decomposition: $A = LL^T$ where L is lower triangular
4. Solve $Ly = X^T W z$ for y (forward substitution)
5. Solve $L^T \hat{\beta} = y$ for $\hat{\beta}$ (backward substitution)

Advantages over LU decomposition: - Exploits SPD structure: $O(p^3/3)$ instead of $O(2p^3/3)$ - Numerically stable for well-conditioned systems - Automatic failure detection: Cholesky fails if matrix is not positive definite

Fallback: If Cholesky fails (due to numerical issues or near-singularity), fall back to LU decomposition with partial pivoting.

Robust log-determinant (`log_det_robust`, `src/linalg.rs`): For the REML cost term $\log|X^T W X + S_\lambda|$, the Cholesky log-det is tried first; on failure (evenly-spaced B-spline design matrices can develop tiny negative floating-point pivots) the symmetric eigensolver `dsyev` is used, clamping eigenvalues to 10^{-300} before summing the logs. This means the REML objective gracefully returns a very negative log-det rather than an error, steering the L-BFGS optimizer away from degenerate λ regions.

1.8.5.2 Covariance Matrix The covariance matrix for $\hat{\beta}$ is:

$$\text{Cov}(\hat{\beta}) = (X^T W X + S_\lambda)^{-1} = A^{-1}$$

With Cholesky $A = LL^T$:

$$A^{-1} = (L^T)^{-1} L^{-1}$$

We compute L^{-1} via triangular inversion, then form $(L^{-1})^T L^{-1}$.

1.8.5.3 Gradient Computation for GCV For GCV optimization, we need $\frac{\partial \text{EDF}}{\partial \lambda_j}$:

$$\frac{\partial \text{EDF}}{\partial \lambda_j} = -\text{tr}(V S_j V X^T W X)$$

where $V = A^{-1}$.

Exploiting block structure: When S_j is block-sparse, only compute:

$$\text{tr}(V[i : i + k, :] S_{\text{block}} V[:, i : i + k] X^T W X)$$

This avoids forming the full $p \times p$ products when $S_{\text{block}} \ll p$.

1.8.6 8.2 REML / Laplace-Approximate Marginal Likelihood

GCV is one option for picking the smoothing parameters; the default `criterion` in `FitConfig` is `Reml` (see `src/fitting/mod.rs:43-54`). REML in this setting is the Laplace-approximate marginal likelihood (LAML) of Wood (2011), evaluated at the working PWLS step. The selector is exposed as a three-variant enum:

```
pub enum SmoothingCriterion { Gcv, Reml /* default */, FellnerSchall }
```

`Gcv` minimizes the score of §8 via L-BFGS on $\log \lambda$. `Reml` minimizes $-V_r$ (below) via L-BFGS on $\log \lambda$. `FellnerSchall` targets the same $-V_r$ via a deterministic multiplicative fixed-point (§8.3).

1.8.6.1 The LAML objective Let $S_\lambda = \sum_j \lambda_j S_j$. The working-model LAML negative log-marginal likelihood is

$$-V_r(\lambda) = \frac{1}{2} \log|X^T W X + S_\lambda| - \frac{1}{2} \log|S_\lambda|_+ + \text{Gaussian PWLS terms (data, } \sigma)$$

where $|\cdot|_+$ denotes the **pseudo-determinant** — the product of strictly positive eigenvalues — required because S_λ is rank-deficient (its null space contains the unpenalized polynomial directions: constants for order-1 penalties, lines for order-2, etc., plus the unpenalized intercept and linear columns).

$\log |X^T W X + S_\lambda|$ is computed via `log_det_robust` (see §8.1): Cholesky on the fast path; symmetric eigensolver fallback for near-PD matrices with tiny negative floating-point pivots (common for evenly-spaced B-spline designs), clamping eigenvalues to 10^{-300} before the log so the REML optimizer naturally avoids degenerate λ regions instead of crashing.

Source: `src/fitting/solver.rs:170-250` (`RemlCost`); `src/linalg.rs` (`log_det_robust`).

1.8.6.2 Pseudo-determinant via grouped block eigendecomposition `penalty_eigen` takes the full list of penalty matrices and their λ values and processes them in **coefficient-block groups**: `penalty_nonzero_block_range` identifies the non-zero row/column range of each penalty, and all penalties that share the exact same range are combined into one group before eigendecomposition.

For each group g :

1. Form the combined scaled block: $B_g = \sum_{j \in g} \lambda_j S_j|_{\text{block}}$
2. Symmetric eigendecompose: $B_g = Q_g \text{diag}(d_1^{(g)}, \dots) Q_g^T$
3. Per-group relative threshold: $\tau_g = \max(\varepsilon \cdot d_{\max}^{(g)}, 10^{-300})$, where $\varepsilon = \text{REML_RANK_TOL_EPS} = 10^{-8}$

Then:

$$\log |S_\lambda|_+ = \sum_g \sum_{d_i^{(g)} > \tau_g} \log d_i^{(g)}, \quad S_\lambda^+ = \text{block-diag}\{B_g^+\}, \quad M_p = \sum_g \text{null_dim}(B_g).$$

Why grouped decomposition? A single global threshold $\tau = \varepsilon \cdot \max(d_{\max}, 1)$ is dominated by the largest- λ term when smoothing parameters span many orders of magnitude (e.g. $\lambda_1 \approx 10^{-8}$ and $\lambda_4 \approx 5 \times 10^{10}$). This misclassifies small- λ non-null directions as null space, flipping the REML gradient sign. The per-group threshold is relative to each block's own spectral norm, eliminating this pollution. The 10^{-300} hard floor prevents collapse when $d_{\max}^{(g)} \approx 0$ (very small λ).

Tensor-product smooths: the two marginal penalties $\lambda_1(S_1 \otimes I_{k_2})$ and $\lambda_2(I_{k_1} \otimes S_2)$ share the same $k_1 k_2$ coefficient block and are therefore combined before eigendecomposition, so the identity $(\lambda_1 S_1 + \lambda_2 S_2)^+ \neq (\lambda_1 S_1)^+ + (\lambda_2 S_2)^+$ is never violated. For disjoint blocks (one smooth per coefficient block) the formula reduces to a per-penalty decomposition.

Source: `src/fitting/solver.rs` (`penalty_eigen`, `penalty_nonzero_block_range`).

1.8.6.3 REML gradient From Wood (2011, §3) the gradient with respect to $\log \lambda_j$ is

$$\frac{\partial(-V_r)}{\partial \log \lambda_j} = \frac{1}{2} \lambda_j \text{tr}(V S_j) - \frac{1}{2} \lambda_j \text{tr}(S_\lambda^+ S_j) - \frac{1}{2} \lambda_j \hat{\beta}^T S_j \hat{\beta} / \hat{\sigma}^2$$

with $V = (X^T W X + S_\lambda)^{-1}$. The implementation evaluates this analytically and feeds it to L-BFGS; final $\log \lambda$ values are clamped to $[-\text{LOG_LAMBDA_CLAMP}, \text{LOG_LAMBDA_CLAMP}] = [-30, 30]$ to keep $\lambda \in [e^{-30}, e^{30}]$.

Source: `src/fitting/solver.rs:214-303`.

1.8.7 8.3 Fellner–Schall Multiplicative Fixed Point

Wood & Fasiolo (2017) show that for the LAML target, the update

$$\lambda_j^{(t+1)} = \lambda_j^{(t)} \cdot \frac{\text{tr}(S_\lambda^+ S_j) - \text{tr}(V S_j)}{\hat{\beta}^T S_j \hat{\beta}}$$

is **monotone non-increasing** in $-V_r$ under mild conditions, has no line search, and converges geometrically near a minimum. The implementation lives at `src/fitting/solver.rs` (`run_optimization_fellner_schall`) and applies three safeguards:

Safeguard	Constant	Value	Role
Relative floor on the numerator $\text{tr}(S_\lambda^+ S_j) - \text{tr}(V S_j)$	FS_NUMERATOR_FLOOR_REL	10^{-12}	Prevents tiny / occasionally negative numerators from collapsing the update.
Floor on the denominator $\hat{\beta}^T S_j \hat{\beta}$	FS_DENOMINATOR_FLOOR	10^{-12}	Avoids division by zero when $\hat{\beta}$ lies in the null space of S_j .
Per-iteration clamp on $\log \lambda_j$	LOG_LAMBDA_CLAMP	± 30	Keeps λ in $[e^{-30}, e^{30}]$, matching the REML L-BFGS clamp.

Loop control: at most `FS_MAX_ITERS` = 50 iterations; converged when

$$\max_j |\log \lambda_j^{(t+1)} - \log \lambda_j^{(t)}| < \text{FS_TOL} = 10^{-4}.$$

The tolerance is set at 10^{-4} (rather than the looser 10^{-3}) because the F-S update is first-order convergent: stopping at 10^{-3} leaves λ still moving by $\approx 0.1\%$ per step, which is measurable on flat LAML landscapes. The extra iterations cost little since each F-S step is a single PWLS solve. Source: `src/fitting/solver.rs` (`FS_TOL`, `FS_MAX_ITERS`).

1.8.7.1 When to pick which criterion

- **GCV**: classical choice, computationally cheap; tends to undersmooth at moderate n and is more prone to multiple local minima on the GCV surface (Reiss & Ogden 2009). The GCV optimizer always runs L-BFGS from the warm-started λ and does not apply an early-exit threshold — a previous skip triggered when the warm-start GCV score was below 10^{-6} could freeze λ for parameters with small residual sums of squares (e.g. a log-scale σ whose working RSS is tiny), preventing those parameters from adapting their smoothness across outer cycles.
- **REML** (default): preferred for stability; the Laplace-approximate marginal likelihood is asymptotically equivalent to true REML for the working PWLS model, and tends to undersmooth less than GCV at moderate sample sizes.
- **FellnerSchall**: same target as REML but with a deterministic multiplicative update — no L-BFGS, no line search. Fast and well-behaved for well-conditioned problems; can stall if the numerator drifts to its floor.

1.8.8 8.4 Collapse-Guarded Restart

Across all three criteria the λ -objective ($-V_r$ or the GCV score) for a single P-spline is **unimodal** in $\log \lambda$, with a single interior optimum, but it flattens into a near-horizontal **shelf** at large λ where the smooth has been driven entirely onto its penalty null space (effective degrees of freedom $\rightarrow$ the null dimension; for an order-2 penalty after centering, a straight line). On that shelf the gradient ≈ 0 and, for Fellner-Schall, $\hat{\beta}^T S_j \hat{\beta} \rightarrow 0$ so the multiplicative ratio explodes upward — both optimizers can become **stuck** there. Because the floating-point reduction order of the dense linear algebra is not deterministic under OpenBLAS, a borderline step can occasionally tip onto the shelf, producing a rare, nondeterministic **collapse** of an otherwise-recoverable smooth.

The selector guards against this in `src/fitting/scoring.rs::step`. After the warm-started optimization and PWLS solve, any smooth term whose EDF has fallen to within `EDF_COLLAPSE_SLACK` = 0.5 of its null dimension is treated as *collapsed*; the optimizer is then re-run once from a deliberately small seed,

$$\lambda_{\text{restart}} = \exp(\log \lambda_{\text{cold}} - \text{RESTART_LOG_OFFSET}), \quad \text{RESTART_LOG_OFFSET} = 8,$$

where $\log \lambda_{\text{cold}}$ is the scale-aware trace-ratio cold start (`initial_log_lambda`). This seed sits well below both the interior optimum and the shelf, so a gradient/fixed-point step descends into the unimodal interior optimum. The incumbent and the restart are then compared by their **actual objective value** (`lambda_cost`), and the lower one is kept. Comparing objectives is what makes the guard safe in both directions: a genuinely null-space-optimal fit (e.g. a strictly linear truth under an order-2 penalty) has the *better* marginal likelihood at the collapsed λ , so its collapse is **preserved**; only a spuriously collapsed, signal-bearing smooth — where the interior fit has the better objective — is repaired. The guard fires only when a collapse is detected, so well-behaved fits are untouched. Source: `src/fitting/scoring.rs` (`step`), `src/fitting/solver.rs` (`restart_seed`, `lambda_cost`, `RESTART_LOG_OFFSET`).

Determinism and the root cause. The *trigger* for a spurious collapse is the nondeterministic reduction order of multi-threaded OpenBLAS: the λ -objective itself is unimodal, so a deterministic fit has no second basin to fall into. This repo therefore pins BLAS to a single thread for all of its own cargo runs (`OPENBLAS_NUM_THREADS=1`, `OMP_NUM_THREADS=1` in `.cargo/config.toml`), which makes the dense linear algebra reproducible. Under single-thread BLAS the control cases (`mu_smooth_recovers_nonlinear_mean_control`, `sigma_smooth_recovers_nonlinear_scale`) recover the interior optimum on **every** repeat (collapse rate 0/20, $\text{corr} \approx 0.9999$ with zero spread — see `tests/lambda_bistability.rs`), so the smooth-recovery tests are gated deterministically in CI rather than left flaky. The collapse-guarded restart above is retained as **defense-in-depth**: it costs nothing on well-behaved fits and still protects any future multi-threaded or otherwise nondeterministic execution path where the tipping could reappear.

1.9 9. Mean-Variance Relationships

A key feature distinguishing different distributions is how variance relates to the mean. This determines which distribution is appropriate for different data structures.

Distribution	Mean	Variance	Relationship
Gaussian	μ	σ^2	Constant (homoskedastic)
Poisson	μ	μ	Equidispersion: $\text{Var} = \text{Mean}$
Gamma	μ	$\mu^2 \sigma^2$	Proportional to μ^2 (constant CV)
Negative Binomial	μ	$\mu + \sigma \mu^2$	Overdispersion relative to Poisson
Binomial	$n\mu$	$n\mu(1 - \mu)$	Quadratic in mean, max at $\mu = 0.5$
Beta	μ	$\frac{\mu(1-\mu)}{1+\phi}$	Max at $\mu = 0.5$, decreases with ϕ
Student-t	μ	$\frac{\nu \sigma^2}{\nu - 2}$	Heavier tails than Gaussian
Ordered Categorical	$\sum_r r \pi_r$	$\sum_r r^2 \pi_r - (\mathbb{E}Y)^2$	Discrete $\{1, \dots, R\}$; depends on all thresholds

1.9.1 Choosing a Distribution

Ordinal responses: - Fixed ordered categories $\{1, \dots, R\}$: Ordered Categorical (`0cat`)

Count data: - Equidispersed: Poisson - Overdispersed (variance $>$ mean): Negative Binomial - Known number of trials: Binomial

Continuous positive: - Constant coefficient of variation: Gamma - Heavy tails: Student-t with ν small - Heteroskedastic with mean: Gamma or lognormal (not implemented)

Proportions/rates in $(0, 1)$: - Beta (continuous) - Binomial (discrete counts / n trials)

Continuous unbounded: - Light tails, constant variance: Gaussian - Heavy tails: Student-t

1.9.2 Overdispersion

Overdispersion occurs when the observed variance exceeds what the distribution predicts from the mean alone.

For count data: - If $\text{Var}(Y) > \mu$: Use Negative Binomial instead of Poisson - The parameter σ in NB quantifies the overdispersion: as $\sigma \rightarrow 0$, NB $\rightarrow$ Poisson

For continuous data: - Gamma allows variance to scale with μ^2 - Student-t allows heavier tails (more extreme values) than Gaussian

1.10 10. Convergence and Initialization

1.10.1 Initialization Strategy

The RS algorithm requires starting values for all parameters. Default initialization:

1. μ :
 - Gaussian: $\bar{y}$ (sample mean)
 - Student-t: $\text{median}(y)$ — a robust location seed. The sample mean is pulled by the heavy tails, biasing the first robustifying weights $w = (\nu + 1)/(\nu + z^2)$.
 - Poisson/Gamma/NB: $\bar{y}$ (then apply log link)
 - Binomial: $\bar{y}/n$ (sample proportion)
 - Beta: $\bar{y}$ (sample mean, clamped to (0.1, 0.9))
2. σ :
 - Gaussian: s_y (sample std dev)
 - Student-t: $1.4826 \cdot \text{MAD}(y)$ — the MAD-to- σ consistency factor for a normal core. A raw sample SD overestimates the scale under heavy tails. Floored at 10^{-4} (falling back to 1.0).
 - Gamma: $\hat{\sigma}_0 = s_y/\bar{y}$ (sample CV), clamped to $[0.05, 10.0]$. σ parameterizes the coefficient of variation, not the raw SD; a raw-SD start makes REML over-penalize the σ smooth on the first RS iteration and warm-start into a full-collapse trap.
 - NB/Beta: 1.0
3. ν (Student-t): 5.0 — a fixed moderate seed, deliberately **not** a sample-kurtosis estimate. For a regression model the *marginal* kurtosis of y reflects the spread of the mean structure rather than the noise tails, so inverting $\kappa = 6/(\nu - 4)$ biases ν ; in the multi-smooth weighted case this biased seed tipped the optimizer into a degenerate over-smoothed basin. 5 sits well clear of the $\nu > 2$ finite-variance boundary. Source: `StudentT::initial_value` in `src/distributions/student_t.rs`.
4. ϕ (Beta): 1.0
5. **Smoothing parameters** λ : initialized from the trace-ratio cold-start heuristic

$$\log \lambda_j^{(0)} = \log \left(\frac{\text{tr}(X^T X)}{\text{tr}(S_j)} \right)$$

per penalty (unweighted, using the assembled X for the current distribution parameter). The all-ones start $\lambda = 1$ can leave $X^T W X + S_\lambda$ near-singular for high-cardinality bases (e.g. $k = 20$) or prior-weighted models where $X^T W X$ is scaled by large prior weights. Source: `src/fitting/mod.rs`, `src/fitting/solver.rs` (`initial_log_lambda`).

1.10.2 Convergence Criterion

The algorithm declares convergence when every distribution parameter θ_k independently satisfies the relative criterion:

$$\frac{\|\beta_k^{(t+1)} - \beta_k^{(t)}\|_\infty}{\max(\|\beta_k^{(t+1)}\|_\infty, 1)} < \epsilon,$$

where k indexes parameters (μ , σ , etc.) and $\epsilon = 10^{-3}$ by default. Each parameter is checked against its **own** coefficient scale rather than a global scale pooled across all parameters. The $\max(\cdot, 1)$ floor makes the test behave like an absolute threshold when all coefficients are $O(1)$ (typical for normalized data).

Using a shared global scale — dividing the maximum change across all parameters by the maximum $|\beta|$ across all parameters — was the previous behaviour. It caused the loop to declare convergence prematurely when one parameter (e.g. μ) had large coefficients and dominated the denominator while another (e.g. log-scale σ) was still drifting in relative terms.

Maximum iterations: 200 (default)

1.10.3 Convergence Issues

Non-convergence can occur due to: 1. **Poor initialization:** Try better starting values, especially for ν in Student-t 2. **Model misspecification:** Wrong distribution family for the data 3. **Multicollinearity:** Highly correlated predictors (check condition number of design matrix) 4. **Insufficient smoothing:** Extremely small λ can cause instability 5. **Numerical issues:** Extreme data values causing overflow/underflow

Diagnostics: - Monitor deviance: should decrease monotonically - Check for NaN/Inf in coefficients or fitted values - Inspect condition number of $(X^T W X + \lambda S)$

1.11 11. Model Selection and Diagnostics

1.11.1 11.1 Information Criteria

Akaike Information Criterion (AIC):

$$\text{AIC} = -2\ell(\hat{\theta}) + 2k$$

where $\ell(\hat{\theta})$ is the maximized log-likelihood and k is the number of parameters.

For GAMLSS with smoothing:

$$k = \sum_{j=1}^P \text{EDF}_j$$

where P is the number of distribution parameters and EDF_j accounts for the effective degrees of freedom of smooth terms.

Bayesian Information Criterion (BIC):

$$\text{BIC} = -2\ell(\hat{\theta}) + k \log(n)$$

BIC penalizes complexity more heavily than AIC for $n > 7$.

1.11.2 11.2 Deviance

The **deviance** measures lack of fit:

$$D = 2[\ell_{\text{saturated}} - \ell_{\text{fitted}}]$$

where $\ell_{\text{saturated}}$ is the log-likelihood of a saturated model (perfect fit).

For nested models, the deviance difference follows approximately χ^2 distribution:

$$D_1 - D_2 \sim \chi^2_{\text{EDF}_2 - \text{EDF}_1}$$

1.11.3 11.3 Residuals

Quantile (randomized quantile) residuals:

$$r_i = \Phi^{-1}(F(y_i|\hat{\theta}_i))$$

where F is the fitted CDF and Φ^{-1} is the inverse standard normal CDF.

If the model is correct, $r_i \sim N(0, 1)$.

Deviance residuals:

$$r_i^{(D)} = \text{sign}(y_i - \hat{\mu}_i) \sqrt{d_i}$$

where d_i is the contribution of observation i to the total deviance.

Pearson residuals:

$$r_i^{(P)} = \frac{y_i - \hat{\mu}_i}{\sqrt{\widehat{\text{Var}}(Y_i)}}$$

1.11.4 11.4 Worm Plots

For each parameter θ_k , plot quantile residuals against fitted quantiles: - X-axis: Normal quantiles $\Phi^{-1}((i - 0.5)/n)$ - Y-axis: Sorted quantile residuals

If the model is correct, points should lie on a horizontal line at zero. Systematic deviations indicate: - U-shape: Underdispersion - Inverse U-shape: Overdispersion - S-shape: Skewness issues - Linear trend: Location shift

1.11.5 11.5 Q-Q Plots

Plot theoretical quantiles against sample quantiles of residuals. Should be approximately linear if residuals are normal.

1.11.6 11.6 Goodness of Link Test

To test if link g is appropriate, fit augmented model:

$$g(\theta) = X\beta + \gamma[g(\theta)]^2$$

Test $H_0 : \gamma = 0$ using likelihood ratio test.

1.12 12. Code Map

A reverse index from mathematical concept to the canonical implementation, for contributors jumping between formula and source.

Concept (this doc)	Source location
Backfitting outer loop with per-parameter convergence (§4.1)	src/fitting/mod.rs (fit_gamlss)
P-IRLS inner step (§4.2)	src/fitting/scoring.rs (step)
MIN_WEIGHT, MAX_STEP (§4.4)	src/fitting/scoring.rs:28,32
Prior / observation weights (§4.5)	src/fitting/scoring.rs (step)
MAX_ETA, MIN_ETA, MIN_POSITIVE (§4.4)	src/distributions/links.rs:8-12
IdentityLink, LogLink, LogitLink (§7)	src/distributions/links.rs:24-59
Gaussian derivatives (§1.1)	src/distributions/gaussian.rs
Student-t derivatives (§1.2)	src/distributions/student_t.rs
Poisson derivatives (§1.3)	src/distributions/poisson.rs
Gamma derivatives (§1.4)	src/distributions/gamma.rs
Negative Binomial derivatives (§1.5)	src/distributions/negative_binomial.rs
Beta derivatives (§1.6)	src/distributions/beta.rs
Binomial derivatives (§1.7)	src/distributions/binomial.rs
Ordered categorical derivatives (§1.8)	src/distributions/ocat.rs
Digamma / trigamma batched (§2)	src/math.rs
Distribution trait (§1)	src/distributions/mod.rs
B-spline basis (de Boor) & uniform knots (§5.0)	src/splines.rs (create_basis_matrix, evaluate_basis_functions_into, select_knots)
Sum-to-zero Householder Z (§5.1)	src/splines.rs (sum_to_zero_basis)
Difference penalty matrix (§5)	src/splines.rs (create_penalty_matrix)
Tensor product assembly (§5)	src/fitting/assembler.rs
Random effect assembly (§5)	src/fitting/assembler.rs
Block-sparse penalty (§5.1)	src/fitting/assembler.rs, src/types/newtypes.rs
PWLS / Cholesky solve (§8.1)	src/fitting/solver.rs (fit_pwls, fit_pwls_with_grad_info)
Robust log-det fallback (§8.1, §8.2)	src/linalg.rs (log_det_robust)
Initial- λ trace-ratio heuristic (§10)	src/fitting/mod.rs, src/fitting/solver.rs (initial_log_lambda)
EDF (§6)	src/fitting/solver.rs (fit_pwls_with_grad_info)
GCV cost and gradient (§8)	src/fitting/solver.rs (GamlssCost)
GCV gradient finite-difference test	src/fitting/solver.rs (gcv_gradient_matches_finite_diff)
GCV L-BFGS driver (§8)	src/fitting/solver.rs (run_optimization)
REML / LAML cost and gradient (§8.2)	src/fitting/solver.rs (RemlCost)
REML gradient finite-difference test	src/fitting/solver.rs (reml_gradient_matches_finite_diff)
Penalty grouped pseudo-determinant / pseudo-inverse (§8.2)	src/fitting/solver.rs (penalty_eigen, penalty_nonzero_block_range)
Fellner-Schall iteration (§8.3)	src/fitting/solver.rs (run_optimization_fellner_schall)
SmoothingCriterion enum (§8.2)	src/fitting/mod.rs
Defaults (max_iterations=200, tolerance=1e-3)	src/fitting/mod.rs:31-32
Diagnostics (AIC/BIC/EDF/residuals, §11)	src/fitting/diagnostics.rs

1.13 13. Notation Glossary

Symbols used throughout this document.

Symbol	Meaning
y, Y	Response value / random variable
n	Number of observations
p	Total number of coefficients across all terms; also (in §5.0) B-spline degree
P	Number of distribution parameters (e.g. $P = 2$ for Gaussian)
θ_k	k -th distribution parameter (μ, σ, ν, ϕ)
μ, σ, ν, ϕ	Location, scale, degrees of freedom (Student-t), precision (Beta)
R	Number of ordered categories in <code>0cat</code> ($R \in \{2, 3, 4, 5\}$)
θ_k	k -th threshold in <code>0cat</code> ($k = 1, \dots, R - 1$); also used generically for the k -th distribution parameter — context distinguishes
δ_k	k -th threshold increment parameter fed to the RS loop: $\theta_1 = \delta_1$ (identity link), $\theta_k = \theta_{k-1} + e^{\delta_k}$ for $k \geq 2$ (log link)
F_k, f_k	Cumulative-logit CDF and logistic density at threshold θ_k : $F_k = \text{logistic}(\theta_k - \mu)$, $f_k = F_k(1 - F_k)$
π_r	Category probability in <code>0cat</code> : $\pi_r = F_r - F_{r-1}$, $F_0 = 0, F_R = 1$
prior_i	Per-observation prior / observation weight (§4.5); defaults to 1
$\text{safe_}w_i$	Floored Fisher weight: $\max(w_i, \text{MIN_WEIGHT})$
g, g^{-1}	Link function and its inverse
$\eta = g(\theta)$	Linear predictor for a distribution parameter
X	Design matrix ($n \times p$); collects intercept, linear, and smooth basis columns
β	Coefficient vector ($p \times 1$)
W	Diagonal weight matrix; $W_{ii} = w_i = \text{Fisher info for observation } i$
$u_i = \partial \ell / \partial \eta_i$	Score (first derivative of log-likelihood on η -scale)
$w_i = -\mathbb{E}[\partial^2 \ell / \partial \eta_i^2]$	Fisher information on η -scale
$z = \eta + u/w$	Working response
S_j	j -th penalty matrix (one per smooth term direction)
λ_j	Smoothing parameter for S_j
$S_\lambda = \sum_j \lambda_j S_j$	Aggregate penalty matrix
$V = (X^T W X + S_\lambda)^{-1}$	PWLS coefficient covariance
$H = X V X^T W$	Hat matrix (projection onto fitted values)
$\text{EDF} = \text{tr}(H) = \text{tr}(V X^T W X)$	Effective degrees of freedom
$\text{RSS} = (z - X \hat{\beta})^T W (z - X \hat{\beta})$	Weighted residual sum of squares
$\psi(x), \psi'(x)$	Digamma, trigamma
$ A _+$	Pseudo-determinant of PSD matrix A (product of strictly positive eigenvalues)
A^+	Moore–Penrose pseudo-inverse of A
$\otimes$	Kronecker product
$\odot$ (in §5)	Row-wise Kronecker product (one row of $B_1 \otimes B_2$ per row index)

Symbol	Meaning
$\mathbf{1}_k$	Column vector of k ones
I_k	$k \times k$ identity matrix

1.14 References

1. Rigby, R. A., & Stasinopoulos, D. M. (2005). Generalized additive models for location, scale and shape. *Journal of the Royal Statistical Society: Series C*, 54(3), 507-554. — Core RS backfitting algorithm and observed-info choice in NB.
2. Eilers, P. H., & Marx, B. D. (1996). Flexible smoothing with B-splines and penalties. *Statistical Science*, 11(2), 89-121. — P-spline difference penalty.
3. de Boor, C. (1978). *A Practical Guide to Splines*. Springer. — Cox-de Boor B-spline recursion.
4. Wood, S. N. (2017). *Generalized Additive Models: An Introduction with R* (2nd ed.). Chapman and Hall/CRC. — Tensor-product smooths, identifiability constraints, GAM reference.
5. Wood, S. N. (2011). Fast stable restricted maximum likelihood and marginal likelihood estimation of semiparametric generalized linear models. *Journal of the Royal Statistical Society: Series B*, 73(1), 3-36. — Laplace-approximate REML for smoothing selection.
6. Wood, S. N., & Fasiolo, M. (2017). A generalized Fellner-Schall method for smoothing parameter optimization with application to Tweedie location, scale and shape models. *Biometrics*, 73(4), 1071-1081. — Multiplicative fixed-point update.
7. Reiss, P. T., & Ogden, R. T. (2009). Smoothing parameter selection for a class of semiparametric linear models. *JRSS B*, 71(2), 505-523. — Comparison of GCV vs REML behavior.
8. Craven, P., & Wahba, G. (1979). Smoothing noisy data with spline functions. *Numerische Mathematik*, 31, 377-403. — Original GCV.
9. McCullagh, P., & Nelder, J. A. (1989). *Generalized Linear Models* (2nd ed.). Chapman and Hall. — IRLS / Fisher scoring derivation underlying §4.3.
10. Abramowitz, M., & Stegun, I. A. (1972). *Handbook of Mathematical Functions*. Dover Publications. — Digamma / trigamma asymptotic expansions (6.3.18, 6.4.11).
11. Piegl, L., & Tiller, W. (1997). *The NURBS Book* (2nd ed.). Springer. — Algorithm A2.2: de Boor-Cox recurrence for stable B-spline evaluation; the canonical reference for the `left/right` array structure used in `evaluate_basis_functions_into`.
12. McCullagh, P. (1980). Regression models for ordinal data. *Journal of the Royal Statistical Society: Series B*, 42(2), 109-142. — Proportional-odds / cumulative-logit model; foundational reference for the `Ucat` distribution (§1.8).